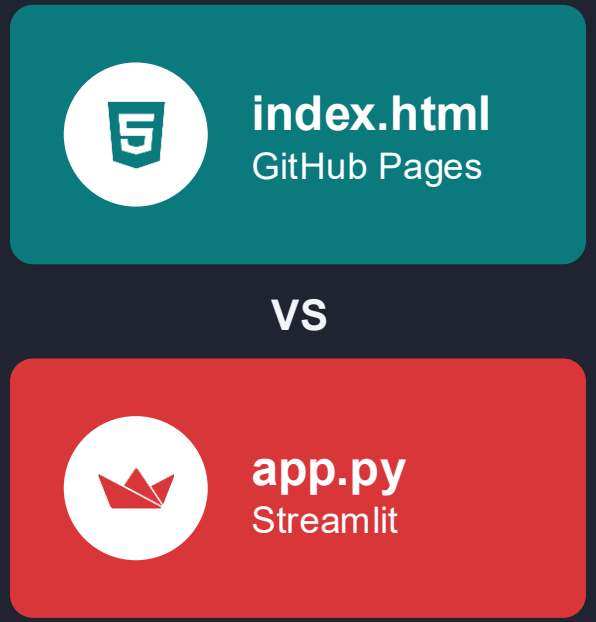
내가 만든 앱, 세상에 공개하기

index.html (GitHub Pages) vs app.py (Streamlit)



디지털리터러시와 영어교육 · 배포(Deploy) 방식 비교

먼저, 배포한다(Deploy)의 의미?



내 컴퓨터

앱을 만들었지만 나만 열어볼 수 있음



인터넷 서버

항상 켜져 있는 컴퓨터에 파일을 올림



친구의 기기

링크만 있으면 누구나 열어볼 수 있음

배포 (deployment) = 내 앱을 인터넷 주소(URL)에 올려서, 링크로 누구나 쓰게 만드는 것

방식 1: GitHub Pages

index.html 파일을 그대로 올려서 웹사이트로 공개



비유: 밀키트 배달

재료와 조리법(HTML·CSS·JavaScript)을 상자째 보내면, 손님(브라우저)이 자기 식탁에서 직접 조리하는 방식. 식탁에서 모든 게 끝나서 주방(서버)은 필요 없음.

방식 2: Streamlit

app.py(Python)를 서버에서 실행해서 결과 화면을 보여줌.



비유: 식당 주방에서 요리

주문(입력)이 들어올 때마다 서버 주방에서 Python으로 요리해서, 완성된 음식(화면)만 내보내는 방식. 복잡한 처리와 외부 서비스 연결도 주방이 담당함.

핵심 차이: 누가 코드를 실행할까?



친구의 브라우저가 실행

- 코드: HTML · CSS · JavaScript
- 받은 파일을 브라우저가 직접 실행
- 버튼에 반응하는 앱은 가능하지만, 서버가 필요한 일은 어려움

비유: 밀키트 · 내 식탁에서 조리



Streamlit 서버가 실행

- 코드: Python
- 서버에서 실행하고 결과 화면만 전달
- 분석·음성·AI 연결 같은 복잡한 처리를 맡음

비유: 식당 · 주방에서 조리

비유로 비교하기

구분	밀키트 (GitHub Pages)	식당 (Streamlit)
조리 장소	손님 식탁 (브라우저)	식당 주방 (서버)
조리 도구	JavaScript만 있음	Python을 쓸 수 있음
선택하면	식탁에서 바로 반응	주문이 주방에 갔다 와서 반응
다른 손님과	각자 식탁에서 혼자	주방은 모두가 함께 씀
비밀 열쇠(API 키)	상자에 들어 있어 손님이 볼 수 있음	주방 금고에 보관

차이는 '선택이 되느냐'가 아니라 '어디서 조리하느냐 ' 로 비유할 수 있음

서버에서 Python이 맡는 일



글 분석

학습자 글의 어휘 수준·문장 길이를 분석



음성 만들기

영어 문장을 읽어 주는 발음 음성을 만들 수 있음



AI 피드백

학생 답을 AI에 보내 피드백을 받음 (API 키는 서버에 보관)



데이터 정리

학생 응답·점수를 표로 모아 정리

무료 서버는 자원 한도가 있어서, 아주 무거운 작업은 외부 AI 서비스에 맡기게 됨.

GitHub Pages 배포 4단계

1

저장소 만들기

Public 저장소 생성

2

파일 올리기

index.html을 Upload files로 올림

3

Pages 켜기

Settings → Pages에서 main 브랜치 선택

4

주소 확인

몇 분 뒤 사이트 주소가 생김

주소 모양: https://아이디.github.io/저장소이름

Streamlit 배포 4단계

1

파일 올리기

app.py와 requirements.txt를 저장소에 생성

2

로그인

share.streamlit.io에서 GitHub로 로그인

3

앱 만들기

Create app → 저장소·브랜치·app.py 선택

4

배포하기

Deploy 클릭 → .streamlit.app 주소가 생김

코드를 고쳐서 GitHub에 올리면 앱이 거의 바로 업데이트돼요

requirements.txt가 하는 일?

저장소 폴더 모양

```
my-app/
|-- app.py
|-- requirements.txt
`-- pages/
```

requirements.txt 안에는 아래처럼 리스트로 적음

```
streamlit
pandas
```

requirements.txt
= 장보기 목록

- 서버는 내 컴퓨터와 달리, 내가 뭘 설치했는지 모르는 새 컴퓨터임.
- 앱이 쓰는 라이브러리(도구)를 목록으로 적어 두면 서버가 알아서 설치.
- 목록에서 빠지면 ModuleNotFoundError로 멈춰요. 가장 흔한 배포 실패 원인!

한눈에 비교하기

구분	GitHub Pages (index.html)	Streamlit (app.py)
사용 언어	HTML · CSS · JavaScript	Python
실행되는 곳	친구의 브라우저	Streamlit 서버
필요한 파일	index.html	app.py + requirements.txt
서버 기능	별도 서버·서비스 필요	Python으로 연결 가능
휴면	없음	12시간 무방문 시 휴면
무료 조건	공개 저장소면 무료	무료 (자원 한도 있음)
잘 맞는 앱	퀴즈·카드 등 화면 중심 앱	분석·음성·AI 연결 앱

장점과 단점

GitHub Pages

장점

- 설정이 가장 간단
- 휴면 없이 항상 열려 있음
- 화면 디자인이 자유로움

단점

- 서버 작업(AI 연결 등)은 어려움
- API 키 같은 비밀을 숨길 수 없음
- 화면 코드가 길어지기 쉬움

Streamlit

장점

- 분석·음성·AI 연결을 Python으로
- API 키를 서버에 보관할 수 있음
- 화면이 자동으로 만들어짐

단점

- 12시간 방문이 없으면 휴면 (zzz)
- 무료라 아주 무거운 작업은 어려움
- requirements.txt 설정 필요

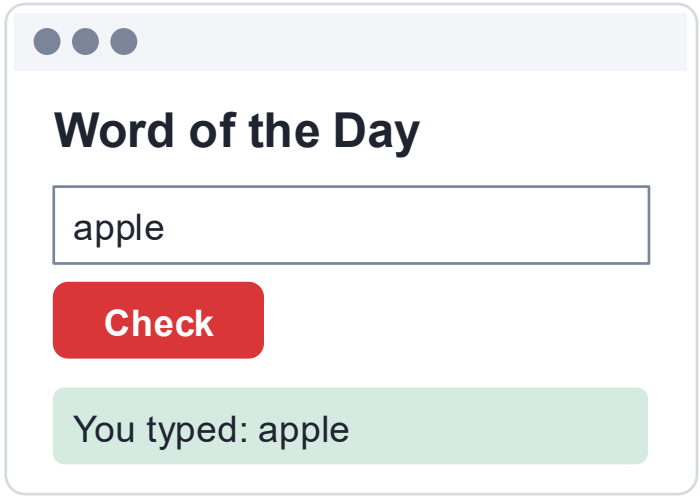
Streamlit의 간단한 코드

app.py (Python 5줄)

```
import streamlit as st

st.title("Word of the Day")
word = st.text_input("English word")
if st.button("Check"):
    st.success(f"You typed: {word}")
```

친구가 보는 화면



입력창·버튼은 몇 줄로 만들고, 복잡한 처리는 Python 라이브러리에 맡기게 됨.

어떤 방식을 고를까?

텍스트 분석·음성·데이터 처리가 필요할 때	→ **Streamlit**
AI에 연결하고 API 키를 숨겨야 할 때	→ **Streamlit**
퀴즈·카드처럼 화면 중심이면 충분해요	→ **GitHub Pages**

기말 프로젝트는 복잡한 처리와 AI 연결을 위해 Streamlit을 중심으로 할 예정임

기말 프로젝트: 왜 Streamlit?



복잡한 처리는 Python 라이브러리가

글 분석·음성 생성·데이터 정리를 몇 줄로 가져다 쓰게 됨



AI 연결을 안전하게

AI 피드백을 받아 오고, API 키는 서버에 보관



여러 앱을 한 홈페이지로

멀티페이지로 학습 앱들을 링크해 홈페이지처럼 사용

Checklist

- [x] app.py 만들기 (막히면 AI에게 도움 요청)
- [x] requirements.txt 만들기 (streamlit 한 줄부터)
- [x] GitHub 저장소(Public)에 파일 올리기
- [x] share.streamlit.io에서 Deploy 누르기
- [x] .streamlit.app 주소를 열어서 확인하고 제출하기

막히면: Manage app → 로그(Logs)의 오류 메시지를 복사해서 AI에게 물어보세요 ☺